

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:
«Операционные системы»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
«Forkbomb»

Выполнил:
Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Проверил:
Ханов Артур Рафаэлевич, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2025г.

Содержание

Анализ воздействия на Linux (Arch).....	4
Исходный код программы Forkbomb.....	5
Исходный код скрипта для логирования и построения графика.....	6

Анализ воздействия на Windows 10

Forkbomb запускалась на 16 ГБ оперативной памяти, 6 ядрах и 12 потоках процессора. Измерения проводились с интервалом времени 0.3 секунды.

При запуске Forkbomb на Windows 10 количество процессов резко возрастает. При этом система намертво не зависает до определённого момента. При запуске несколько раз поведение ОС было очень похожим: сначала начинались сильные тормоза в интерфейсе, и потом в определённый момент система завершает все созданные процессы. Максимальное количество процессов, которое удалось замерить — около 5000.

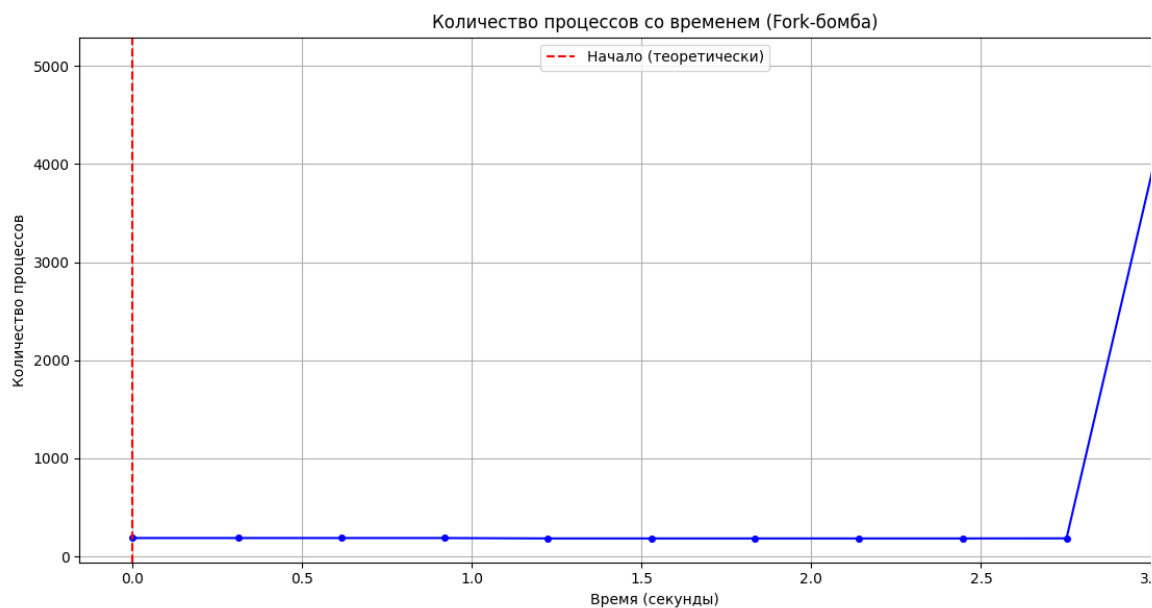


Рисунок 1: График количества процессов Windows

Анализ воздействия на Linux (Arch)

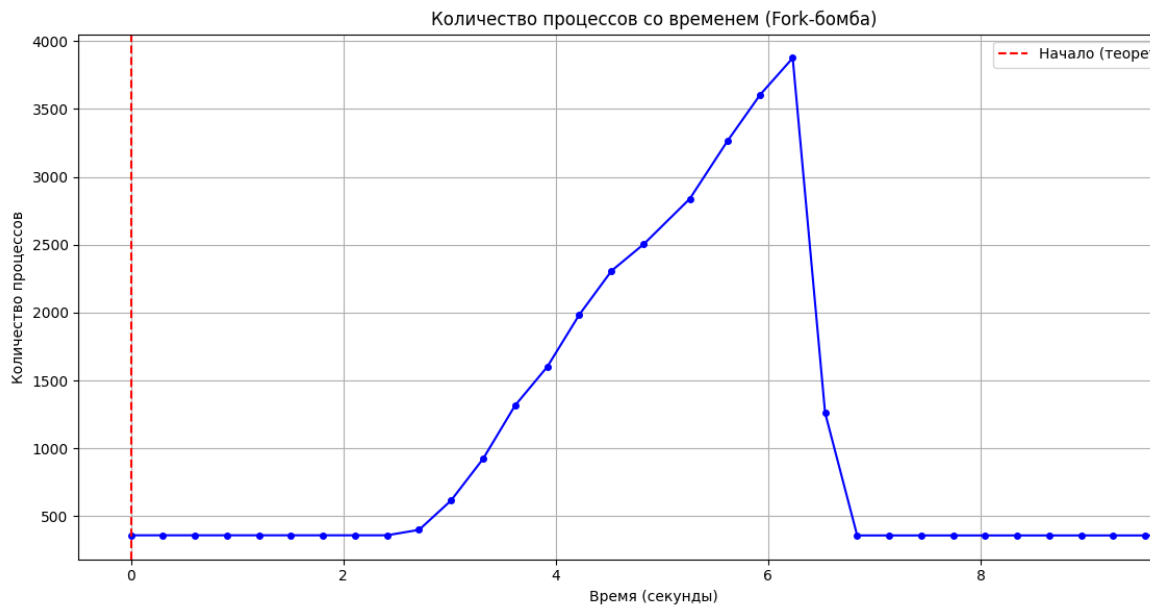


Рисунок 2: График количества процессов Linux

При запуске программы график очень резко начинает возрастать. В определенном момент времени выполнение программы было остановлено вручную и количество процессов практически мгновенно вернулось к исходному состоянию. Если этого не сделать, то система достигает около 17-18 тысяч процессов и зависает намертво. Помогает только хард-релоад. Но в Linux есть способ борьбы с Forkbomb путём ограничения количества процессов для каждого пользователя используя `ulimit -u` или правку файла `/etc/security/limits.conf`.

Исходный код программы Forkbomb

forkbomb.py:

```
import subprocess
import sys
import os

print("Запуск forkbomb")

while True:
    try:
        subprocess.Popen([sys.executable, __file__])
    except Exception as e:
        if hasattr(os, 'fork'):
            os.fork()
```

Исходный код скрипта для логирования и построения графика

measurements.py:

```
import psutil
import time
import matplotlib.pyplot as plt
import os
import datetime

INTERVAL = 0.3
DURATION = 10

def count_processes():
    return len(psutil.pids())

process_counts = []
timestamps = []
start_time = time.time()

curr_time = 0
while curr_time < DURATION:
    count = count_processes()
    process_counts.append(count)
    timestamps.append(curr_time)
    print(f"Время: {int(curr_time)}с, Процессов: {count}")
    time.sleep(INTERVAL)
    curr_time = time.time() - start_time

plt.figure(figsize=(12, 6))
plt.plot(timestamps, process_counts, marker='o', linestyle='--', color='b',
markersize=4)
plt.title('Количество процессов со временем (Fork-бомба)')
plt.xlabel('Время (секунды)')
plt.ylabel('Количество процессов')
plt.grid(True)
plt.axvline(x=0, color='r', linestyle='--', label='Начало (теоретически)')
plt.legend()
plt.tight_layout()

timestamp_str = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
filename = f"graph_{timestamp_str}.png"
plt.savefig(filename)
print(f"График сохранен как {filename}")
plt.show()
```